# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 3 – CODING CHALLENGE

---

| | | | |
|---|---|---|---|
| **Course Code:** | DSA03 | **Course Title:** | Natural Language Processing |
| **CO Assessed:** | CO1 – Imitation (L1) | **Assessment:** | Assessment Tool 3 – Coding Challenge |
| **Weightage:** | 30% of CIA | | |
| **CO5 Statement:** | Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3) | | |
| **Student Name:** | M Vaishnavi | **Reg. No.:** | 192324162 |
| **Section / Batch:** | 2023 batch | **Date:** | |

**Code of Conduct**

*I M Vaishnavi (192324162) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: vaishnavi

**Instructions**

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

| Section / Topic | Focus | Q Nos |
|---|---|---|
| **Regular Expression Pattern Matching** | Regular Expressions, Basic Regular Expression Patterns | 1 |
| **Finite State Automata Implementation** | Finite State Automata, Models and Algorithms | 2 |
| **Advanced Regular Expression Search** | Regular Expressions, Basic Regular Expression Patterns | 3 |

## Annexure A – Coding Challenge

### Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

**Develop a Python application that validates:**
Email Address
Password
Mobile Number

using Regular Expressions.

**Validation Rules**
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
**Password**
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
**Mobile Number**
Must contain exactly 10 digits.
First digit should be between 6–9.
**Expected Output**

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

### Section 2: Design a Finite State Automata (FSA) Simulator

2. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

**The program should:**

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state
    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

**Example**

Language:
Strings ending with "ab"

**Input**
abaab

**Output**
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

## Section 3: Smart Pattern Matching Engine using Regular Expressions

### 3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

**The system should support:**
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

**Example Input**
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

**User Menu**
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

**Expected Output**
Display all matching patterns based on user selection.

## Rubrics for Calculation

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| **Problem Understanding** | 20% | Clearly understands problem constraints, edge cases, and competitive requirements | Understands problem with minor gaps in constraints | Partial understanding; misses key constraints | Misinterprets problem or constraints |
| **Algorithm** | 25% | Selects optimal algorithm with best time and space complexity for competitive constraints | Uses efficient algorithm with minor scope for improvement | Uses basic/inefficient approach | No clear algorithmic approach |
| **Code Implementation** | 20% | Writes correct, efficient, and clean code that passes all constraints | Code works with minor errors or inefficiencies | Partially correct code; fails for some cases | Code does not execute or gives incorrect results |
| **Competitive Performance** | 20% | Solves problem within time limits and handles large inputs effectively | Solves within acceptable time with minor delays | Struggles with time limits or larger inputs | Unable to solve within time constraints |
| **Test Case Handling** | 15% | Handles all test cases including edge and hidden cases | Handles most test cases | Misses important edge cases | Fails on multiple test cases |

| Q. No. / Criteria | Problem Understanding | Algorithm | Code Implementation | Competitive Performance | Test Case Handling | Sub. Total | Total % |
|---|---|---|---|---|---|---|---|
| 1 | | | | | | | |
| 2 | | | | | | | |
| 3 | | | | | | | |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

# Answers – Coding Challenge Solutions (DSA03 – Natural Language Processing)

## Question 1: Intelligent Email and Password Validator using Regular Expressions

### Problem Statement

Develop a Python application that validates an email address, a password and a mobile number against predefined security policies using Regular Expressions, and displays Valid Email / Invalid Email, Strong Password / Weak Password and Valid Mobile Number / Invalid Mobile Number.

### Approach

- Each field is validated by its own small function using a pre-compiled regular expression (modular design).
- re.fullmatch() is used so that the whole string must satisfy the rule, not just a part of it.
- The password policy uses look-ahead assertions, one for each rule (uppercase, lowercase, digit, special character), followed by {8,} for the minimum length.
- password_feedback() lists exactly which rules a weak password has failed, so the result can be justified.
- Empty or non-string input raises a ValueError, which is handled in the program. The program runs interactively or with the --demo option for the test cases.

### Regular Expression Patterns Used

| Field | Pattern | Explanation |
|---|---|---|
| Email | `^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(?:com\|org\|edu\|net\|in)$` | Starts with a letter; letters, digits, "." and "_" before "@"; domain of letters only; extension must be .com, .org, .edu, .net or .in |
| Password | `^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$` | Look-aheads require an uppercase letter, a lowercase letter, a digit and one of @ # $ % & !; .{8,} gives at least 8 characters |
| Mobile | `^[6-9]\d{9}$` | First digit 6-9 followed by exactly 9 more digits (10 digits in total) |

### Python Program

```
"""
Q1 - Intelligent Email and Password Validator using Regular Expressions
DSA03 - Natural Language Processing | CO1 Assessment Tool 3
"""
import re
import sys

# ---------------- Compiled regular expression patterns ----------------
# Email: starts with a letter, then letters/digits/'.'/'_' before '@',
# domain made only of letters, extension from the allowed list.
EMAIL_PATTERN = re.compile(r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(?:com|org|edu|net|in)$")

# Password: four look-aheads (upper, lower, digit, special) + minimum length 8.
PASSWORD_PATTERN = re.compile(
    r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$"
)

# Mobile: exactly 10 digits, first digit between 6 and 9.
MOBILE_PATTERN = re.compile(r"^[6-9]\d{9}$")


def _check_text(value):
    """Reject non-string / empty input before any pattern is applied."""
    if not isinstance(value, str) or not value.strip():
        raise ValueError("Input is empty or not a string")
    return value.strip()
```

```
def validate_email(email):
    return "Valid Email" if EMAIL_PATTERN.fullmatch(_check_text(email)) else "Invalid Email"


def validate_password(password):
    # passwords may legitimately contain spaces, so they are not stripped
    if not isinstance(password, str) or not password:
        raise ValueError("Input is empty or not a string")
    return "Strong Password" if PASSWORD_PATTERN.fullmatch(password) else "Weak Password"


def password_feedback(password):
    """List the policy rules that a weak password fails."""
    rules = [
        (r".{8,}", "minimum 8 characters"),
        (r"[A-Z]", "an uppercase letter"),
        (r"[a-z]", "a lowercase letter"),
        (r"\d", "a digit"),
        (r"[@#$%&!]", "a special character (@ # $ % & !)"),
    ]
    return [msg for pat, msg in rules if not re.search(pat, password)]


def validate_mobile(mobile):
    ok = MOBILE_PATTERN.fullmatch(_check_text(mobile))
    return "Valid Mobile Number" if ok else "Invalid Mobile Number"


def run_demo():
    tests = [
        ("rahul.sharma@gmail.com", "Rahul@123", "9876543210"),
        ("1rahul@gmail.com",       "rahul123",  "5876543210"),
        ("priya_n@simats.edu",     "Priya#2026", "8123456789"),
        ("arun@mail123.com",       "Ab1@",       "98765432"),
        ("sneha.reddy@company.xyz", "STRONG@PASS1", "98765432101"),
        ("kumar_k@yahoo.in",       "Kumar&2026", "7012345678"),
    ]
    for n, (e, p, m) in enumerate(tests, 1):
        print(f"Test Case {n}")
        print(f"  Email    : {e:<26} -> {validate_email(e)}")
        res = validate_password(p)
        miss = password_feedback(p)
        print(f"  Password : {p:<26} -> {res}")
        if miss:
            print(f"             missing: {', '.join(miss)}")
        print(f"  Mobile   : {m:<26} -> {validate_mobile(m)}")
        print()
    print("Error handling check:")
    try:
        validate_email("   ")
    except ValueError as err:
        print("  ValueError caught ->", err)


def interactive():
    print("=== User Registration Validator ===")
    try:
        email = input("Enter email    : ")
        password = input("Enter password : ")
        mobile = input("Enter mobile   : ")
        print(validate_email(email))
        print(validate_password(password))
        print(validate_mobile(mobile))
    except ValueError as err:
        print("Error:", err)
```

```
if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--demo":
        run_demo()
    else:
        interactive()
```

## Sample Output (python q1_validator.py --demo)

```
Test Case 1
  Email    : rahul.sharma@gmail.com     -> Valid Email
  Password : Rahul@123                  -> Strong Password
  Mobile   : 9876543210                 -> Valid Mobile Number

Test Case 2
  Email    : 1rahul@gmail.com           -> Invalid Email
  Password : rahul123                   -> Weak Password
             missing: an uppercase letter, a special character (@ # $ % & !)
  Mobile   : 5876543210                 -> Invalid Mobile Number

Test Case 3
  Email    : priya_n@simats.edu         -> Valid Email
  Password : Priya#2026                 -> Strong Password
  Mobile   : 8123456789                 -> Valid Mobile Number

Test Case 4
  Email    : arun@mail123.com           -> Invalid Email
  Password : Ab1@                       -> Weak Password
             missing: minimum 8 characters
  Mobile   : 98765432                   -> Invalid Mobile Number

Test Case 5
  Email    : sneha.reddy@company.xyz    -> Invalid Email
  Password : STRONG@PASS1               -> Weak Password
             missing: a lowercase letter
  Mobile   : 98765432101                -> Invalid Mobile Number

Test Case 6
  Email    : kumar_k@yahoo.in           -> Valid Email
  Password : Kumar&2026                 -> Strong Password
  Mobile   : 7012345678                 -> Valid Mobile Number

Error handling check:
  ValueError caught -> Input is empty or not a string
```

## Output Justification

- **Test Case 1:** All three inputs follow every rule, so Valid Email, Strong Password and Valid Mobile Number are displayed.
- **Test Case 2:** The email starts with a digit; the password has no uppercase letter and no special character; the mobile number starts with 5.
- **Test Case 3:** .edu is an allowed extension and Priya#2026 contains upper, lower, digit and special character.
- **Test Case 4:** The domain "mail123" contains digits; the password has only 4 characters; the mobile number has only 8 digits.
- **Test Case 5:** .xyz is not an allowed extension; the password has no lowercase letter; 11 digits fail the "exactly 10 digits" rule.
- **Test Case 6:** "_" is allowed before "@", .in is allowed, and "&" is an accepted special character, so all three are valid.
- **Error handling:** A blank input raises ValueError instead of being silently reported as invalid.

## Question 2: Finite State Automata (DFA) Simulator

### Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator that accepts the DFA description (states, input alphabet, transition table, initial state, final states), accepts multiple input strings, simulates the state transitions and displays whether each string is Accepted or Rejected.

### Approach

- A DFA class stores the 5-tuple M = (Q, Σ, δ, q0, F); the transition function δ is a dictionary {(state, symbol): next_state}.
- _validate() checks that the description is well formed: start and final states belong to Q, every transition uses known states and symbols, and δ is defined for every (state, symbol) pair.
- run() reads the string symbol by symbol, records every state visited (the transition path) and accepts only if the last state is in F. A symbol outside Σ rejects the string.
- read_dfa() lets the user type the DFA and any number of input strings; run_demo() simulates two built-in DFAs.

### DFA Used for the Example: strings over {a, b} ending with "ab"

| Component | Value |
|---|---|
| States (Q) | { q0, q1, q2 } |
| Input alphabet (Σ) | { a, b } |
| Initial state (q0) | q0 |
| Final state(s) (F) | { q2 } |
| Meaning of the states | q0 = start / no progress;  q1 = last symbol read was "a";  q2 = last two symbols read were "ab" |

| State | On a | On b |
|---|---|---|
| → q0 | q1 | q0 |
| q1 | q1 | q2 |
| * q2  (final) | q1 | q0 |

### Python Program

```
"""
Q2 - Deterministic Finite Automata (DFA) Simulator
DSA03 - Natural Language Processing | CO1 Assessment Tool 3
"""
import sys


class DFA:
    def __init__(self, states, alphabet, transitions, start, finals):
        self.states = set(states)
        self.alphabet = set(alphabet)
        self.transitions = dict(transitions)      # {(state, symbol): next_state}
        self.start = start
        self.finals = set(finals)
        self._validate()

    def _validate(self):
        """Make sure the DFA description is well formed."""
        if self.start not in self.states:
            raise ValueError(f"Initial state '{self.start}' is not in the state set")
        if not self.finals <= self.states:
```

```
            raise ValueError("Final state(s) must be a subset of the states")
        for (s, a), t in self.transitions.items():
            if s not in self.states or t not in self.states:
                raise ValueError(f"Transition ({s},{a}) -> {t} uses an unknown state")
            if a not in self.alphabet:
                raise ValueError(f"Transition symbol '{a}' is not in the alphabet")
        for s in self.states:                       # DFA must be complete
            for a in self.alphabet:
                if (s, a) not in self.transitions:
                    raise ValueError(f"Missing transition for state '{s}' on symbol '{a}'")

    def run(self, text):
        """Simulate the DFA. Returns (path, accepted, error_message)."""
        state, path = self.start, [self.start]
        for ch in text:
            if ch not in self.alphabet:
                return path, False, f"symbol '{ch}' is not in the alphabet"
            state = self.transitions[(state, ch)]
            path.append(state)
        return path, state in self.finals, None

    def show_table(self):
        syms = sorted(self.alphabet)
        print(f"  {'State':<8}" + "".join(f"{a:<8}" for a in syms))
        for s in sorted(self.states):
            mark = ("->" if s == self.start else "  ") + ("*" if s in self.finals else " ")
            print(f"  {mark}{s:<6}" + "".join(f"{self.transitions[(s, a)]:<8}" for a in syms))


def simulate(dfa, strings):
    for w in strings:
        path, ok, err = dfa.run(w)
        print(f"Input : '{w}'")
        print("Transition Path:", " -> ".join(path))
        if err:
            print(f"Rejected ({err})")
        else:
            print("Accepted" if ok else "Rejected")
        print()


def read_dfa():
    """Read the DFA description from the user."""
    states = input("States (comma separated)          : ").replace(" ", "").split(",")
    alphabet = input("Input alphabet (comma separated)  : ").replace(" ", "").split(",")
    start = input("Initial state                      : ").strip()
    finals = input("Final state(s) (comma separated)  : ").replace(" ", "").split(",")
    print("Enter transitions as  state,symbol,next_state  (blank line to finish)")
    trans = {}
    while True:
        line = input("  > ").strip()
        if not line:
            break
        s, a, t = [x.strip() for x in line.split(",")]
        trans[(s, a)] = t
    return DFA(states, alphabet, trans, start, finals)


def run_demo():
    # DFA 1 : strings over {a,b} that end with "ab"
    ends_ab = DFA(
        states=["q0", "q1", "q2"], alphabet=["a", "b"],
        transitions={("q0", "a"): "q1", ("q0", "b"): "q0",
                     ("q1", "a"): "q1", ("q1", "b"): "q2",
                     ("q2", "a"): "q1", ("q2", "b"): "q0"},
        start="q0", finals=["q2"])
    print("DFA 1 : strings ending with 'ab'   (* = final state, -> = start)")
```

```
    ends_ab.show_table()
    print()
    simulate(ends_ab, ["abaab", "abba", "ab", "bbb", "abc"])

    # DFA 2 : binary strings containing an even number of 1s
    even_ones = DFA(
        states=["e", "o"], alphabet=["0", "1"],
        transitions={("e", "0"): "e", ("e", "1"): "o",
                     ("o", "0"): "o", ("o", "1"): "e"},
        start="e", finals=["e"])
    print("DFA 2 : binary strings with an even number of 1s")
    even_ones.show_table()
    print()
    simulate(even_ones, ["1010", "111", "000"])

    print("Error handling check:")
    try:
        DFA(["q0"], ["a"], {}, "q9", ["q0"])
    except ValueError as err:
        print("  ValueError caught ->", err)


def interactive():
    try:
        dfa = read_dfa()
        words = input("Input strings (comma separated): ").replace(" ", "").split(",")
        print()
        simulate(dfa, words)
    except (ValueError, KeyError) as err:
        print("Error:", err)


if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--demo":
        run_demo()
    else:
        interactive()
```

## Sample Output (python q2_dfa.py --demo)

```
DFA 1 : strings ending with 'ab'   (* = final state, -> = start)
  State   a       b
  -> q0    q1      q0
     q1    q1      q2
    *q2    q1      q0

Input : 'abaab'
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted

Input : 'abba'
Transition Path: q0 -> q1 -> q2 -> q0 -> q1
Rejected

Input : 'ab'
Transition Path: q0 -> q1 -> q2
Accepted

Input : 'bbb'
Transition Path: q0 -> q0 -> q0 -> q0
Rejected

Input : 'abc'
Transition Path: q0 -> q1 -> q2
Rejected (symbol 'c' is not in the alphabet)
```

```
DFA 2 : binary strings with an even number of 1s
  State   0       1
  ->*e     e       o
     o     o       e

Input : '1010'
Transition Path: e -> o -> o -> e -> e
Accepted

Input : '111'
Transition Path: e -> o -> e -> o
Rejected

Input : '000'
Transition Path: e -> e -> e -> e
Accepted

Error handling check:
  ValueError caught -> Initial state 'q9' is not in the state set
```

## Output Justification

- **abaab:** q0 -a-> q1 -b-> q2 -a-> q1 -a-> q1 -b-> q2. The last state q2 is final, so the string is Accepted.
- **abba:** ends in "ba", the machine finishes in q1 which is not final, so it is Rejected. "bbb" never reaches q2, so it is Rejected.
- **ab:** the shortest accepted string, q0 -a-> q1 -b-> q2.
- **abc:** "c" is not in the alphabet, so the simulation stops and the string is Rejected with a message.
- **DFA 2:** 1010 has two 1s (even) and 000 has none, so both end in state e and are Accepted; 111 has three 1s and ends in state o, so it is Rejected.
- **Note on the sample path in the question:** a 5-symbol input always visits 6 states (the start state plus one per symbol). The program therefore prints q0 → q1 → q2 → q1 → q1 → q2 for "abaab", and the result is Accepted as expected.
- **Error handling:** An invalid DFA description (for example initial state q9 that is not in Q) raises a ValueError with a clear message.

## Question 3: Smart Pattern Matching Engine using Regular Expressions

### Problem Statement

Develop a Python-based text search engine that uses Regular Expressions to support word, prefix, suffix, date, phone number, hashtag and mention (@username) searches through a user menu, and displays all matching patterns for the selected option.

### Approach

- One small function per search type, each using a regular expression; the patterns for dates, phone numbers, hashtags and mentions are pre-compiled.
- Prefix, suffix and word searches build the pattern from the keyword using re.escape() so that special characters typed by the user are treated as plain text, and use re.IGNORECASE for case-insensitive searching.
- A menu-driven loop (handle_choice) runs the search chosen by the user and prints all matches, or "No match found". Invalid menu choices and empty keywords are handled.

### Regular Expression Patterns Used

| Search | Pattern | Explanation |
|---|---|---|
| Date | `\b(?:0?[1-9]\|[12]\d\|3[01])[/-](?:0?[1-9]\|1[0-2])[/-]\d{4}\b` | dd/mm/yyyy or dd-mm-yyyy with day 1-31, month 1-12 and a 4-digit year |
| Phone number | `(?<!\d)[6-9]\d{9}(?!\d)` | 10 digits starting with 6-9, not part of a longer number |
| Hashtag | `(?<!\w)#\w+` | "#" followed by word characters |
| Mention | `(?<!\w)@\w+` | "@" followed by a username (ignores the @ inside e-mail addresses) |
| Prefix | `\b<prefix>\w*` | Words that begin with the keyword (case-insensitive) |
| Suffix | `\b\w*<suffix>\b` | Words that end with the keyword (case-insensitive) |
| Word | `\b<word>\b` | Whole-word match of the keyword (case-insensitive) |

### Python Program

```
"""
Q3 - Smart Pattern Matching Engine using Regular Expressions
DSA03 - Natural Language Processing | CO1 Assessment Tool 3
"""
import re
import sys

SAMPLE_TEXT = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing"""

# ---------------- Compiled regular expression patterns ----------------
DATE_PATTERN    = re.compile(r"\b(?:0?[1-9]|[12]\d|3[01])[/-](?:0?[1-9]|1[0-2])[/-]\d{4}\b")
PHONE_PATTERN   = re.compile(r"(?<!\d)[6-9]\d{9}(?!\d)")
HASHTAG_PATTERN = re.compile(r"(?<!\w)#\w+")
MENTION_PATTERN = re.compile(r"(?<!\w)@\w+")


def search_date(text):
    return DATE_PATTERN.findall(text)
```

```
def search_phone(text):
    return PHONE_PATTERN.findall(text)


def search_hashtag(text):
    return HASHTAG_PATTERN.findall(text)


def search_mention(text):
    return MENTION_PATTERN.findall(text)


def search_word(text, word):
    return re.findall(r"\b" + re.escape(word) + r"\b", text, re.IGNORECASE)


def search_prefix(text, prefix):
    """Words that START with the given prefix."""
    return re.findall(r"\b" + re.escape(prefix) + r"\w*", text, re.IGNORECASE)


def search_suffix(text, suffix):
    """Words that END with the given suffix."""
    return re.findall(r"\b\w*" + re.escape(suffix) + r"\b", text, re.IGNORECASE)


def show(title, matches):
    print(f"  {title}: {matches if matches else 'No match found'}")


def handle_choice(choice, text, arg=None):
    if choice == "1":   show("Dates", search_date(text))
    elif choice == "2": show("Phone numbers", search_phone(text))
    elif choice == "3": show("Hashtags", search_hashtag(text))
    elif choice == "4": show("Mentions", search_mention(text))
    elif choice == "5": show(f"Words with prefix '{arg}'", search_prefix(text, arg))
    elif choice == "6": show(f"Words with suffix '{arg}'", search_suffix(text, arg))
    elif choice == "7": show(f"Word '{arg}'", search_word(text, arg))
    else:
        print("  Invalid choice - please select 1 to 7")


MENU = """
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit"""


def run_demo():
    print("Input text:")
    for line in SAMPLE_TEXT.splitlines():
        print("   ", line)
    print(MENU)
    steps = [("1", None), ("2", None), ("3", None), ("4", None),
             ("5", "nat"), ("6", "ing"), ("7", "CALL"), ("5", "xyz"), ("9", None)]
    for choice, arg in steps:
        print(f"\nEnter your choice: {choice}" + (f"   (keyword: {arg})" if arg else ""))
        handle_choice(choice, SAMPLE_TEXT, arg)
    print("\nEnter your choice: 0\nExiting...")
```

```
def interactive():
    print("Enter the text (finish with an empty line):")
    lines = []
    while True:
        line = input()
        if not line:
            break
        lines.append(line)
    text = "\n".join(lines) or SAMPLE_TEXT
    while True:
        print(MENU)
        choice = input("Enter your choice: ").strip()
        if choice == "0":
            break
        arg = input("Enter keyword: ").strip() if choice in ("5", "6", "7") else None
        if choice in ("5", "6", "7") and not arg:
            print("  Keyword cannot be empty")
            continue
        handle_choice(choice, text, arg)


if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "--demo":
        run_demo()
    else:
        interactive()
```

## Sample Output (python q3_search.py --demo)

```
Input text:
    Meeting on 12/09/2026
    Call 9876543210
    #NLP
    @OpenAI
    natural language processing

1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
7.Search Word
0.Exit

Enter your choice: 1
  Dates: ['12/09/2026']

Enter your choice: 2
  Phone numbers: ['9876543210']

Enter your choice: 3
  Hashtags: ['#NLP']

Enter your choice: 4
  Mentions: ['@OpenAI']

Enter your choice: 5   (keyword: nat)
  Words with prefix 'nat': ['natural']

Enter your choice: 6   (keyword: ing)
  Words with suffix 'ing': ['Meeting', 'processing']

Enter your choice: 7   (keyword: CALL)
  Word 'CALL': ['Call']
```

```
Enter your choice: 5   (keyword: xyz)
  Words with prefix 'xyz': No match found

Enter your choice: 9
  Invalid choice - please select 1 to 7

Enter your choice: 0
Exiting...
```

## Output Justification

- **Options 1-4:** return 12/09/2026, 9876543210, #NLP and @OpenAI, exactly the date, phone number, hashtag and mention present in the input text.
- **Option 5 (prefix "nat"):** only "natural" begins with "nat".
- **Option 6 (suffix "ing"):** "Meeting" and "processing" end with "ing", including the capitalised word at the start of a line.
- **Option 7 (word "CALL"):** matches "Call" because the search is case-insensitive.
- **No match / invalid choice:** prefix "xyz" prints "No match found", and choice 9 prints an error message instead of stopping the program.